



Academic Year: 2026-27

Semester: V

Class / Branch: TEIT

Subject: MDM (Embedded System & RTOS)

Topic:

2.2. Case Study: ARM Cortex-M3 Features, Architecture, Programmer's model, SpecialRegisters, Operating Modes and States, MPU, Memory map and NVIC.

ARM Cortex-M3:

The ARM Cortex-M3 is a 32-bit RISC processor developed by ARM. It is specifically designed for embedded systems that require high performance, low power consumption, and low cost. The ARM Cortex-M3 is a Harvard architecture, 3-stage pipeline, and Thumb-2 instruction set. It includes a configurable Nested Vectored Interrupt Controller (NVIC) and an optional Memory Protection Unit (MPU) for high-performance, low-power real-time embedded control applications. It is widely used in:

- Industrial automation
- Medical devices
- Automotive electronics
- Consumer electronics
- IoT devices
- Robotics

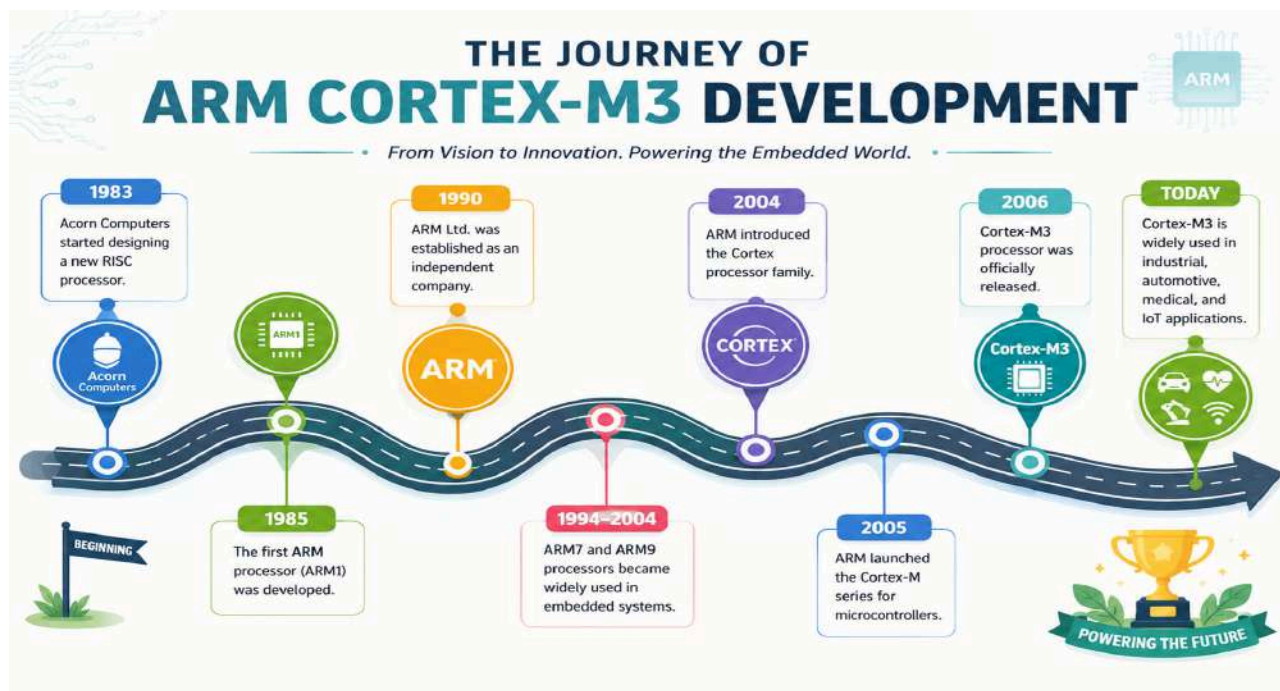


Academic Year: 2026-27

Semester: V

Class / Branch: TEIT

Subject: MDM (Embedded System & RTOS)



Year	Development
1983	Acorn Computers started designing a new RISC processor.
1985	The first ARM processor (ARM1) was developed.
1990	ARM Ltd. was established as an independent company.
1994–2004	ARM7 and ARM9 processors became widely used in embedded systems.
2004	ARM introduced the Cortex processor family.
2005	ARM launched the Cortex-M series for microcontrollers.
2006	Cortex-M3 processor was officially released.
Today	Cortex-M3 is widely used in industrial, automotive, medical, and IoT applications.

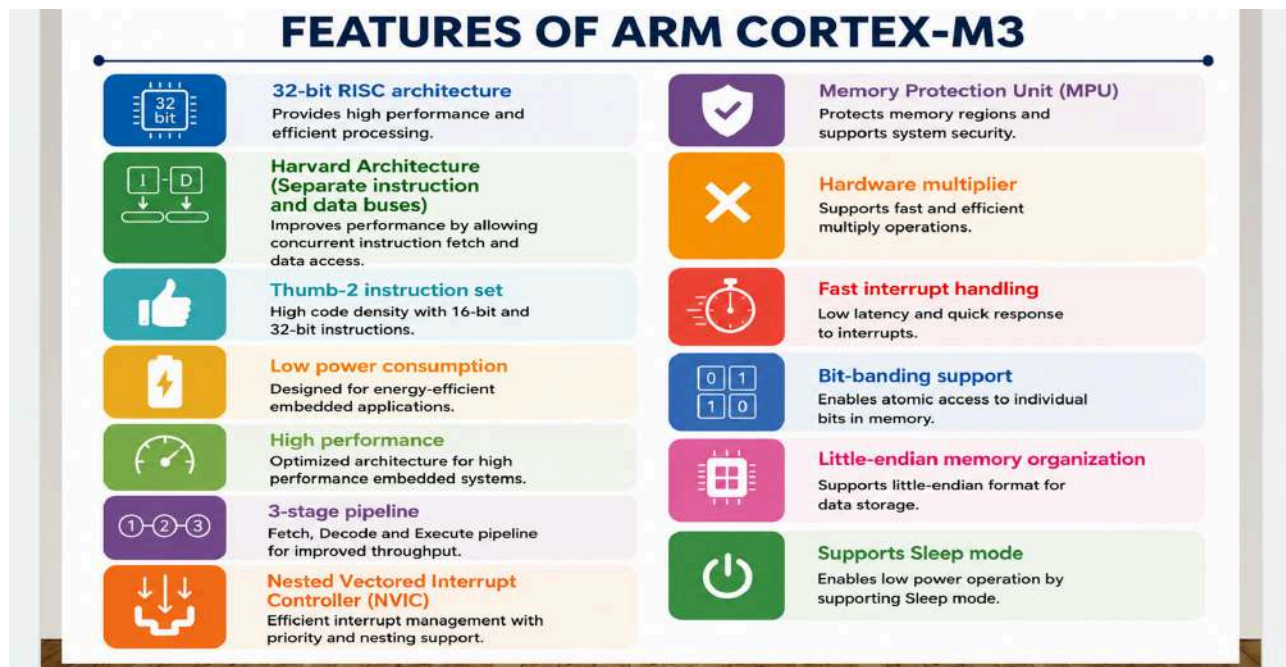


Academic Year: 2026-27

Class / Branch: TEIT

Semester: V

Subject: MDM (Embedded System & RTOS)



1. 32-bit RISC Architecture

- Cortex-M3 uses a 32-bit processor architecture.
- RISC (Reduced Instruction Set Computer) uses a smaller and simpler set of instructions.
- This allows instructions to execute quickly and provides good performance with low complexity.

Example: It can process 32-bit data efficiently in a single operation.

2. Harvard Architecture

- Cortex-M3 uses separate paths/buses for instruction and data access.



Academic Year: 2026-27

Semester: V

Class / Branch: TEIT

Subject: MDM (Embedded System & RTOS)

- The processor can fetch an instruction and access data simultaneously.
- This improves overall processing speed.

Simple idea:

Instruction Memory → CPU ← Data Memory

3. Thumb-2 Instruction Set

- Thumb-2 combines 16-bit and 32-bit instructions.
- 16-bit instructions provide high code density, reducing program memory usage.
- 32-bit instructions provide more powerful operations.

Benefit: Good balance between performance and memory efficiency.

4. Low Power Consumption

- Cortex-M3 is designed for embedded systems that operate with limited power.
- It provides power-saving modes such as Sleep mode.

Applications: Sensors, IoT devices, portable devices, etc.

5. High Performance

- The architecture is optimized to execute instructions efficiently.
- It provides good performance for real-time embedded applications.

6. 3-Stage Pipeline

The processor divides instruction processing into three stages:

Fetch → Decode → Execute

1. Fetch: Get the instruction from memory.
2. Decode: Understand what the instruction means.
3. Execute: Perform the required operation.

Because different instructions can be in different stages at the same time, throughput increases.



Academic Year: 2026-27

Semester: V

Class / Branch: TEIT

Subject: MDM (Embedded System & RTOS)

7. Nested Vectored Interrupt Controller (NVIC)

- NVIC manages interrupts in the Cortex-M3.
- It supports interrupt prioritization.
- Higher-priority interrupts can be handled before lower-priority ones.
- It also supports nested interrupts.

Example: If a low-priority task is running and an important sensor interrupt occurs, the processor can temporarily handle the sensor interrupt.

8. Memory Protection Unit (MPU)

- MPU protects different regions of memory.
- It can control which part of memory a program is allowed to access.
- This improves system reliability and security.

Example: Preventing one program from accidentally modifying another program's memory.

9. Hardware Multiplier

- Cortex-M3 has hardware support for multiplication.
- Multiplication can therefore be performed much faster than implementing it using many software instructions.

Useful in: Signal processing, control systems, mathematical calculations, etc.

10. Fast Interrupt Handling

- Cortex-M3 is designed for quick interrupt response.
- This is important in real-time systems where the processor must respond immediately to external events.

Example: A timer interrupt can trigger an operation at a precise time.

11. Bit-Banding Support

- Bit-banding allows individual bits in certain memory regions to be accessed atomically.
- Instead of reading and modifying an entire byte/word, a single bit can be manipulated.



Academic Year: 2026-27

Semester: V

Class / Branch: TEIT

Subject: MDM (Embedded System & RTOS)

Example:

Changing one control bit from:

$0 \rightarrow 1$

without affecting the other bits.

12. Little-Endian Memory Organization

- In little-endian format, the least significant byte (LSB) is stored at the lowest memory address.

13. Sleep Mode Support

- Cortex-M3 can enter Sleep mode when the CPU is not required.
- This reduces power consumption.
- The processor can wake up when an interrupt or other event occurs.

Example:

Active $\rightarrow$ Sleep $\rightarrow$ Interrupt $\rightarrow$ Wake up $\rightarrow$ Active



Academic Year: 2026-27

Semester: V

Class / Branch: TEIT

Subject: MDM (Embedded System & RTOS)

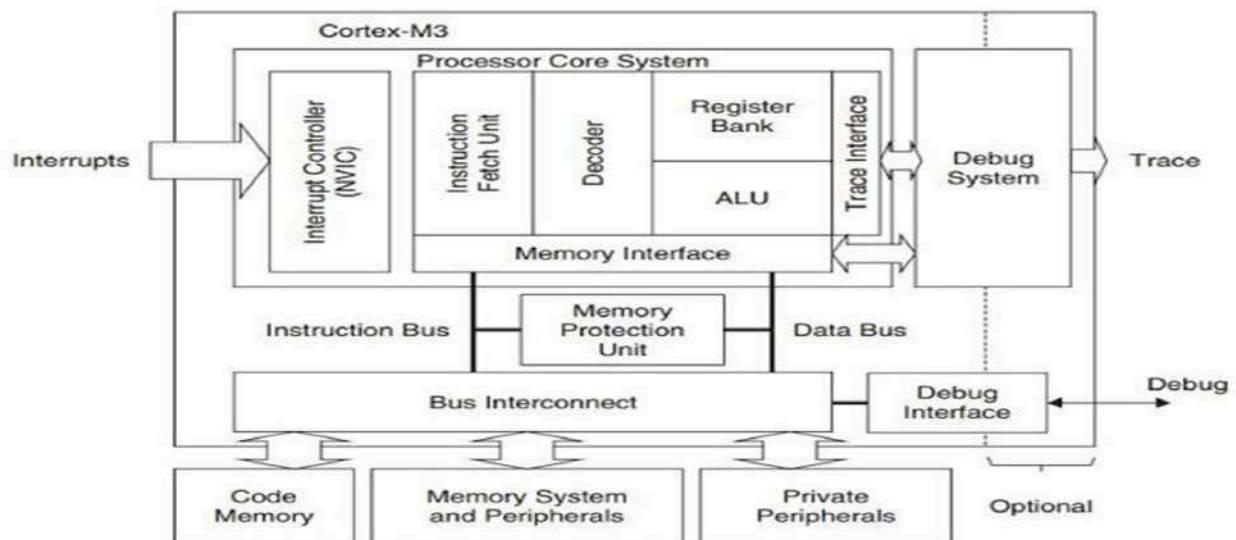


Fig.Architecture ARM Cortex M3

It explains how different hardware blocks work together to execute a program, access memory, communicate with peripherals, and handle interrupts.

1. Interrupt Controller (NVIC)

The NVIC (Nested Vectored Interrupt Controller) is responsible for managing all interrupts generated by peripherals.

Functions

- Receives interrupt requests.
- Assigns priority to interrupts.
- Allows nested interrupts.
- Automatically saves and restores CPU registers.



Academic Year: 2026-27

Semester: V

Class / Branch: TEIT

Subject: MDM (Embedded System & RTOS)

Working

Suppose the CPU is executing a program.

Example

Suppose

- Keyboard Interrupt → Priority 2
- Fire Alarm Interrupt → Priority 1

Fire alarm is more important.

CPU immediately switches to Fire Alarm ISR.

2. Processor Core System

The processor core is the heart (brain) of Cortex-M3. It contains

- Instruction Fetch Unit
- Decoder
- Register Bank
- ALU
- Memory Interface

Every instruction passes through these blocks.

3. Instruction Fetch Unit

This unit fetches instructions from Flash (Code Memory).

4. Decoder

The fetched instruction is machine language.

The Decoder converts it into control signals.

Example:



Academic Year: 2026-27

Semester: V

Class / Branch: TEIT

Subject: MDM (Embedded System & RTOS)

Instruction

ADD R1,R2,R3

Decoder understands

- Operation = ADD
- Operand1 = R2
- Operand2 = R3
- Destination = R1

5. Register Bank

Registers are very fast memory locations inside CPU.

Cortex-M3 contains:

R0-R12-General Purpose Registers

R13-Stack Pointer

R14-Link Register

R15-Program Counter

Example

R0 = 20

R1 = 30

ADD R2,R0,R1

After execution

R2 = 50

Registers make execution very fast.

6.. ALU (Arithmetic Logic Unit)

The ALU performs all calculations.

Arithmetic Operations

- Addition
- Subtraction
- Multiplication



Academic Year: 2026-27

Semester: V

Class / Branch: TEIT

Subject: MDM (Embedded System & RTOS)

Logical Operations

- AND
- OR
- XOR
- NOT

Shift Operations

- Left Shift
- Right Shift

Example:

$$15 + 20 = 35$$

ALU performs this calculation.

10. Memory Protection Unit (MPU)

MPU protects memory.

It divides memory into different regions.

Example:

- Flash
- Read Only
- SRAM
- Read/Write
- Peripheral
- Read/Write

If user program tries to modify Flash

Access Denied

↓

Memory Fault

11. Bus Interconnect

Think of Bus Interconnect as a traffic controller.

Every request goes through Bus Interconnect.

CPU wants Flash

↓

Bus Interconnect

↓



Academic Year: 2026-27

Semester: V

Class / Branch: TEIT

Subject: MDM (Embedded System & RTOS)

Flash
CPU wants UART
↓
Bus Interconnect
↓
UART

It ensures proper communication.

■ Bus Interconnect

Think of Bus Interconnect as a traffic controller.

Every request goes through Bus Interconnect.

CPU wants Flash
↓
Bus Interconnect
↓
Flash
CPU wants UART
↓
Bus Interconnect
↓
UART

It ensures proper communication.

12. Code Memory

Code Memory stores:

- Program
- Constants



Academic Year: 2026-27

Semester: V

Class / Branch: TEIT

Subject: MDM (Embedded System & RTOS)

Usually Flash ROM.

Stored inside Flash.

Flash is:

- Non-volatile
- Retains data after power OFF

13. Memory System and Peripherals

This includes:

- SRAM
- GPIO
- UART
- SPI
- I²C
- ADC
- Timers
- PWM



Academic Year: 2026-27

Semester: V

Class / Branch: TEIT

Subject: MDM (Embedded System & RTOS)

14. Private Peripherals

Private peripherals are built into Cortex-M3.

Examples:

- SysTick Timer
- NVIC
- System Control Block (SCB)

These peripherals are used internally by CPU

15. Debug Interface

Debug Interface connects external debugger.

Examples:

- JTAG
- SWD (Serial Wire Debug)

Functions:

- Program download
- Single step execution
- Breakpoints
- Memory viewing

16. Debug System

Debug System helps programmer during software development.

It allows

- Pause CPU
- Resume CPU



Academic Year: 2026-27

Semester: V

Class / Branch: TEIT

Subject: MDM (Embedded System & RTOS)

- Monitor Registers
- Monitor Memory
- Detect Errors

Without disturbing normal execution.

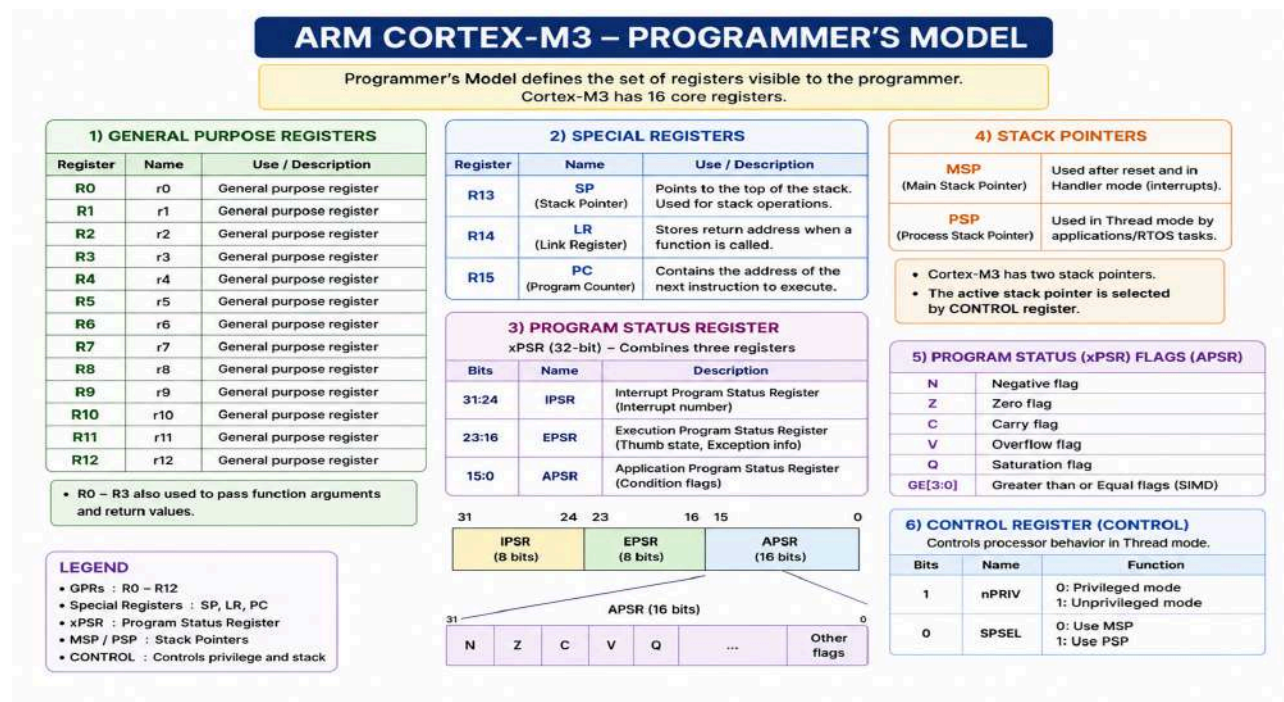
17. Trace Interface

Trace Interface records CPU activity.

It shows:

- Executed instructions
- Function calls
- Interrupt occurrence
- Exceptions

Useful for finding software bugs.





Academic Year: 2026-27

Semester: V

Class / Branch: TEIT

Subject: MDM (Embedded System & RTOS)

NVIC (Nested Vectored Interrupt Controller) in ARM Cortex-M3 :

NVIC (Nested Vectored Interrupt Controller) is a hardware interrupt controller built inside the ARM Cortex-M3 processor. Its main job is to receive interrupt requests from peripherals, decide which interrupt should be served first, and quickly transfer control to the correct Interrupt Service Routine (ISR). It is tightly integrated with the CPU, which provides very low interrupt latency.

Why is NVIC Needed?

Imagine a microcontroller connected to many devices:

- Keyboard
- Timer
- UART
- ADC
- GPIO
- SPI

All of these devices can generate interrupts.

Without NVIC:

- CPU would not know which interrupt to handle first.
- Multiple interrupts could create conflicts.
- Interrupt handling would be slow.

NVIC solves these problems by managing interrupts automatically.

Main Functions of NVIC :

1. Interrupt Management

NVIC receives interrupt requests from peripherals.

Example:



Academic Year: 2026-27

Semester: V

Class / Branch: TEIT

Subject: MDM (Embedded System & RTOS)

- Timer overflow
- UART receives data
- Push button pressed

Each of these sends an interrupt signal to NVIC.

2. Priority Management

Every interrupt can have a priority.

If Timer and Fire Alarm occur together, NVIC executes Fire Alarm first.

3. Nested Interrupts

This is the reason for the word Nested.

Suppose:

- Timer ISR is executing.
- Suddenly UART interrupt occurs.
- UART has higher priority.

NVIC pauses the Timer ISR. This is called Interrupt Nesting.

Memory Protection Unit

The Memory Protection Unit (MPU) is a hardware component in the ARM Cortex-M3 that controls which parts of memory can be accessed by software and what type of access is allowed.

- MPU acts like a security guard for memory. It prevents a program from accessing memory areas that it is not supposed to access.
- If a faulty program accidentally tries to modify the program code, the MPU can detect that the access is not permitted and generate a MemManage fault.
- The MPU divides the memory address space into regions.
- Each region can be configured with different permissions.



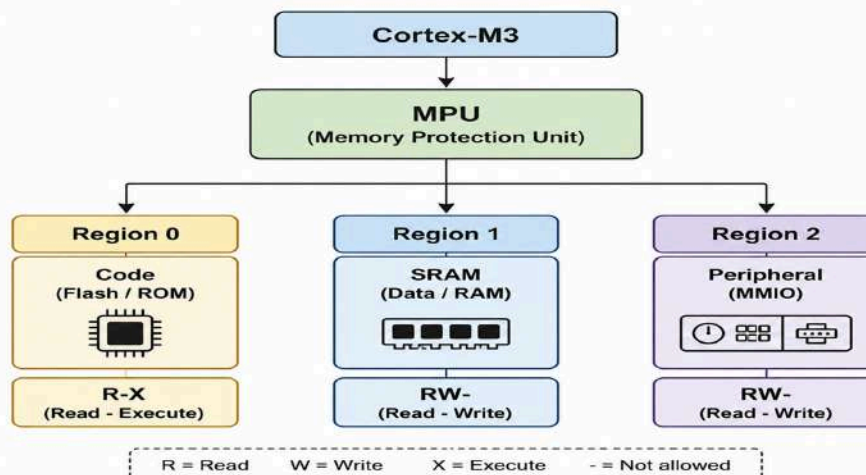
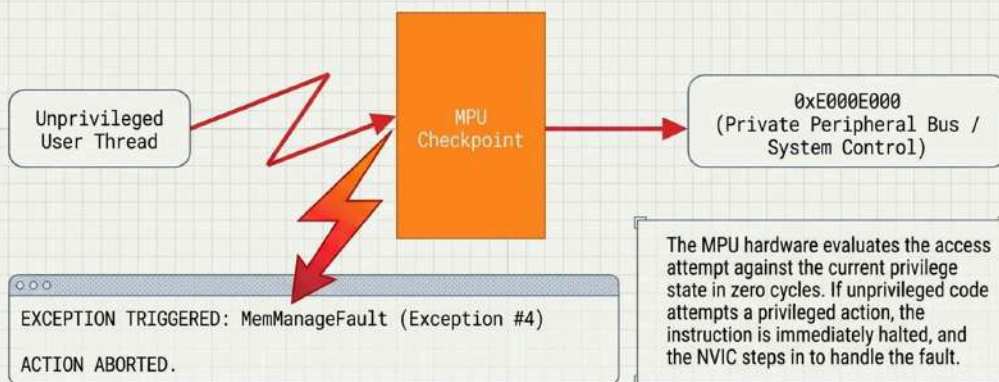
Academic Year: 2026-27

Semester: V

Class / Branch: TEIT

Subject: MDM (Embedded System & RTOS)

The MPU Forcefield: MemManageFault





Academic Year: 2026-27

Semester: V

Class / Branch: TEIT

Subject: MDM (Embedded System & RTOS)

MPU ACCESS PERMISSIONS

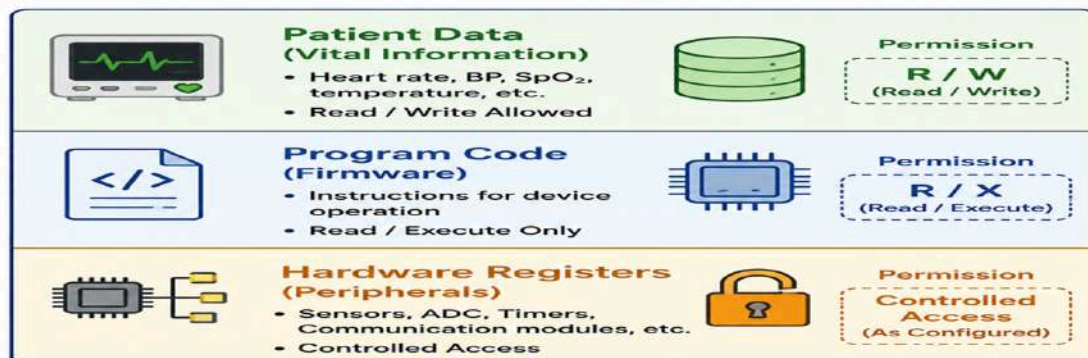
The MPU can control whether a region is:

PERMISSION	MEANING	SYMBOL	EXAMPLE
READ 	Data can be read	R	CPU can read data from this region 
WRITE 	Data can be modified	W	CPU can write or modify data 
EXECUTE 	Instructions can be executed	X	CPU can execute instructions from this region 
NO ACCESS 	Access is prohibited	—	CPU is not allowed to read, write or execute 

R = Read W = Write X = Execute — = Not allowed

MPU in Real-Life Example

Medical Monitoring Device



MPU



Prevents Illegal Memory Access

- ✓ Stops unauthorized read / write / execute
- ✓ Generates fault on violation and protects system



Academic Year: 2026-27

Semester: V

Class / Branch: TEIT

Subject: MDM (Embedded System & RTOS)

Memory Map:

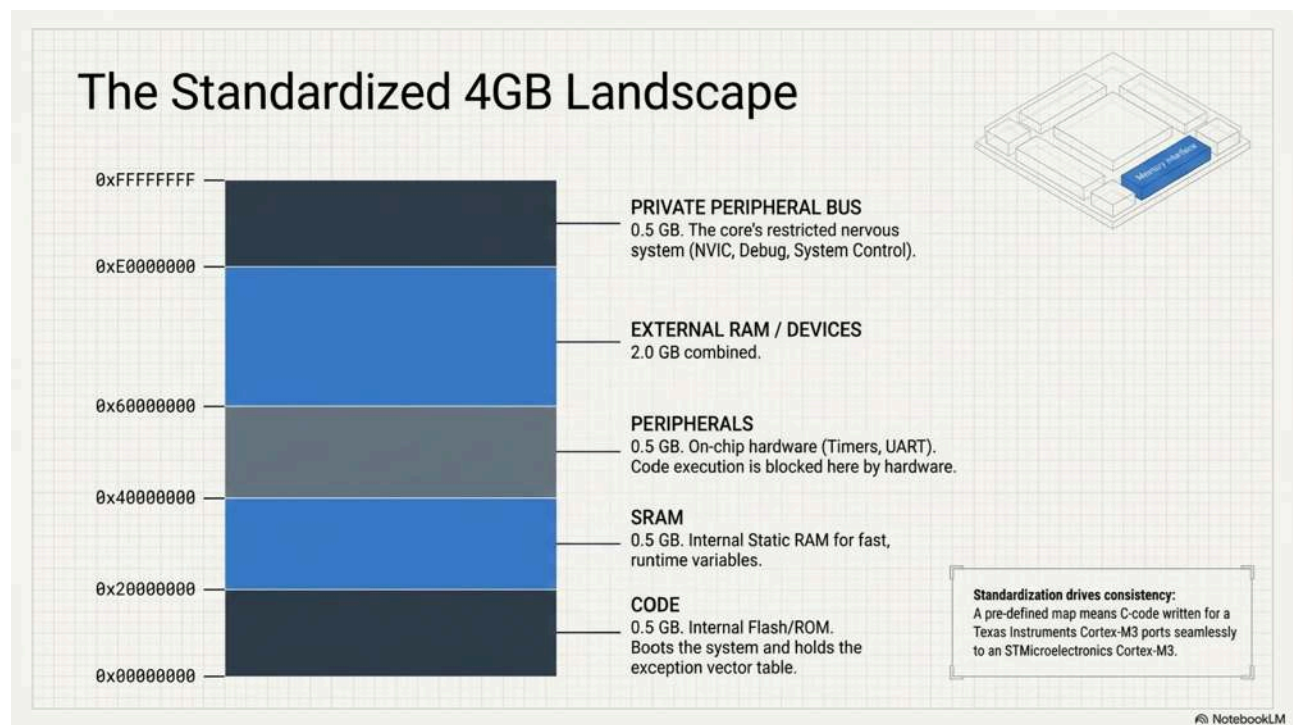
The memory map of the Arm Cortex-M3 defines which address range is used for which type of memory or peripheral.

Cortex-M3 uses a 32-bit address space, so it can address:

$2^{32} = 4 \text{ GB}$ of address space.

The Cortex-M3 has a 4 GB address space. Different parts of this space are assigned to different things such as:

- Program (Code)
- Data (SRAM)
- Peripherals
- System control





Academic Year: 2026-27

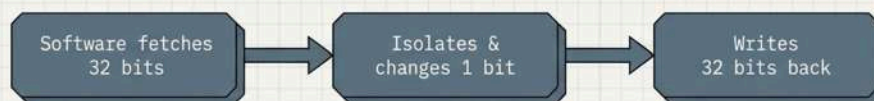
Semester: V

Class / Branch: TEIT

Subject: MDM (Embedded System & RTOS)

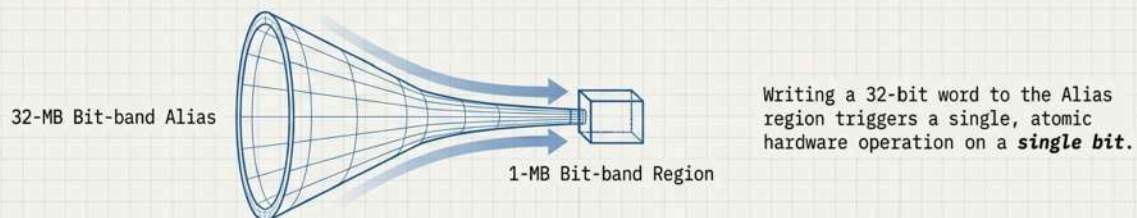
Bit-Banding: Bypassing the Software Bottleneck

Traditional Read-Modify-Write



⚠ Slow & risks interruption

Cortex-M3 Bit-Banding



Benefit: Instant Boolean manipulation for hardware peripherals (like flipping a GPIO pin) without software overhead.

© NotebookLM

Eradicating Overhead with Tail-Chaining

Scenario: IRQ 1 and IRQ 2 arrive almost simultaneously.



When IRQ 1 finishes, the NVIC sees IRQ 2 pending. Instead of unstacking and re-stacking memory, it skips it entirely, jumping straight to the next handler in just 6 cycles.

© NotebookLM



PARSHVANATH CHARITABLE TRUST'S

A. P. SHAH INSTITUTE OF TECHNOLOGY

Department of Information Technology

(NBA Accredited)



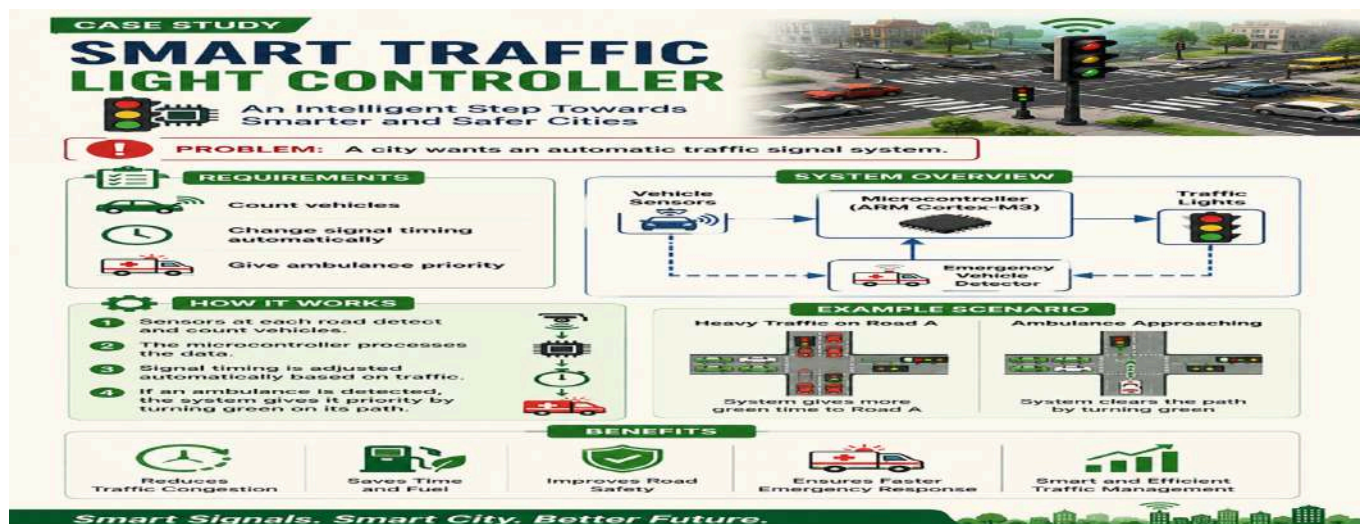
Academic Year: 2026-27

Semester: V

Class / Branch: TEIT

Subject: MDM (Embedded System & RTOS)

ARM A-SERIES vs R-SERIES vs M-SERIES				
THREE POWERFUL FAMILIES. THREE DIFFERENT MISSIONS.				
FEATURE	A-SERIES Application Profile	R-SERIES Real-Time Profile	M-SERIES Microcontroller Profile	
MAIN GOAL	High computing performance	Fast & predictable response	Low-cost, low-power embedded control	
PERFORMANCE	★★★★★ Very High	★★★★☆ High	★★★☆☆ Moderate - High	
REAL-TIME RESPONSE	★★★☆☆ Good (not main focus)	★★★★★ Excellent	★★★★☆ Very Good	
POWER CONSUMPTION	Higher	Medium	Very Low	
MEMORY REQUIREMENT	Large	Medium to Large	Small to Medium	
MEMORY MANAGEMENT	Usually MMU	MPU / Protection features (Core dependent)	Usually MPU, No MMU	
INTERRUPT HANDLING	Good	★★★★★ Extremely Deterministic	★★★★★ Fast	
OPERATING SYSTEM	Linux, Android, Windows, etc.	RTOS / Specialized Real-Time OS	Bare-metal / RTOS	
TYPICAL APPLICATION	Smartphones, Tablets, Laptops, Servers	Automotive Control, Industrial Control, Storage Systems	IoT Devices, Sensors, Home Appliances, Wearables	
EXAMPLE CORES	Cortex-A53, A72, A76, A78, A710	Cortex-R5, R7, R52	Cortex-M0, M0+, M3, M4, M7, M33	
SYSTEM COST	\$\$\$\$ Higher	\$\$\$ Medium - High	\$ Lower	
SAFETY FOCUS	General-purpose computing	★★★★★ Safety & Reliability	★★★★☆ Embedded Reliability	
BEST CHOICE WHEN...	You need MAXIMUM PERFORMANCE	You need GUARANTEED TIMING	You need SIMPLE, EFFICIENT CONTROL	
EASY WAY TO REMEMBER				
	A = Application Focus on PERFORMANCE	R = Real-Time Focus on PREDICTABILITY	M = Microcontroller Focus on EFFICIENCY	CORTEX-M3 BELONGS TO M-SERIES Optimized for low power, cost-effective embedded and microcontroller applications with fast interrupt handling.
★ ONE ARCHITECTURE. THREE PURPOSES. ENDLESS POSSIBILITIES. ★				





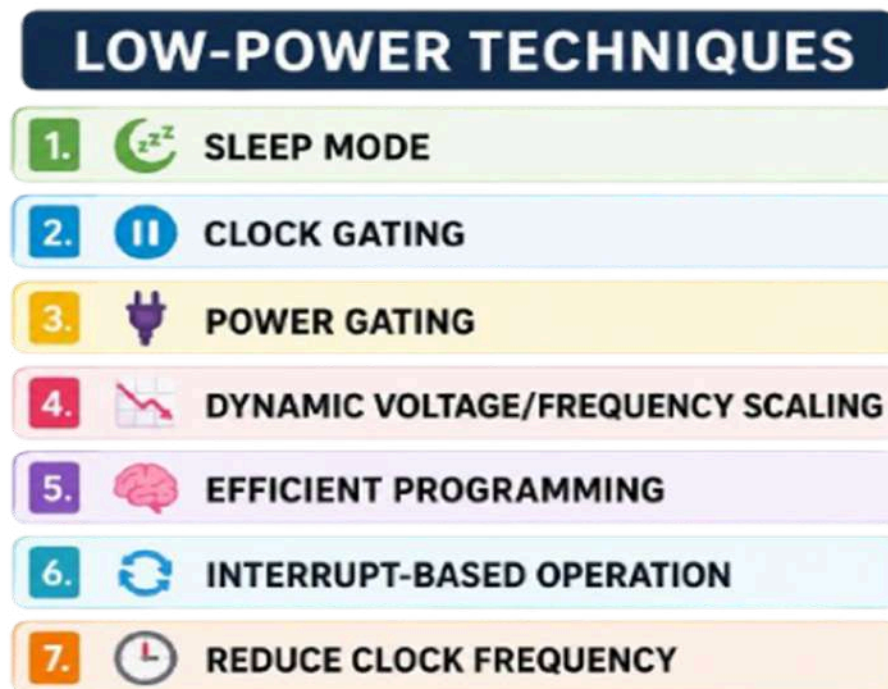
Academic Year: 2026-27

Semester: V

Class / Branch: TEIT

Subject: MDM (Embedded System & RTOS)

Low power: - Need and techniques:



1. Sleep Mode

Stop the CPU when it has no work to do while keeping necessary peripherals active.

2. Clock Gating

Turn off the clock to unused peripherals or modules.

Clock

| CPU ON

| Timer ON

| UART OFF

| ADC OFF



Academic Year: 2026-27

Semester: V

Class / Branch: TEIT

Subject: MDM (Embedded System & RTOS)

3. Power Gating

Remove power from unused sections of the system.

4. Dynamic Voltage/Frequency Scaling

Reduce voltage and clock frequency when high performance is not required.

5. Efficient Programming

Reduce unnecessary calculations, memory accesses, and continuous polling.

6. Interrupt-Based Operation

Instead of continuously checking a device:

7. Reduce Clock Frequency

A lower clock frequency generally reduces dynamic power consumption.